ZoneOpt.jl: Agentic Optimization of Ice Hockey Zone Entry Strategies via Neural ODEs

Greta Reitenbach
18.337 / 6.7320 Parallel Computing and Scientific Machine Learning

Abstract

In competitive ice hockey, transitioning the puck across the offensive blue line represents a critical inflection point in possession and scoring probability. Traditional sports analytics treat spatial transitions, such as carrying, dumping, or passing the puck into the zone, as discrete classification problems. ZoneOpt.jl is a custom, hardware-accelerated Julia framework designed to model and optimize these tactical zone entries as continuous dynamical systems. Formulated as a Universal Differential Equation (UDE), the framework merges physical priors (kinematics and dampening drag) with a Neural Network that discovers unmodeled dynamics (player collisions, stick deflections, ice friction variation) directly from high-frequency spatial tracking data. The project highlights memory-efficient backpropagation using the Continuous Adjoint Method, optimized Vector-Jacobian Products via Reverse-Mode Automatic Differentiation, execution of data-parallel EnsembleProblem abstractions, and efficient sensitivity analysis for initial-condition optimization using Forward-Mode Differentiation.

1. Introduction and Motivation

In modern hockey analytics, the transition from the neutral zone to the offensive zone dictates the flow of play. While “carrying” the puck into the zone is statistically linked to higher shot volumes, the severe risk of turnovers at the blue line often leads teams to “dump” the puck (shooting it into the far boards behind the net), conceding immediate possession for a safer defensive posture. Classical statistical models, such as static Random Forests or basic sequence classifiers, fail to capture the continuous, high-speed physical realities of these movements or the spatial influence of defensive geometries.

A hockey play is fundamentally a continuous dynamical system governed by rigid body physics intersecting with human agency. Scientific Machine Learning offers an ideal paradigm for modeling this environment. By treating the puck's motion as a neural-augmented ordinary differential equation, ZoneOpt.jl achieves three critical milestones:

1. It accurately interpolates time irrespective of the tracking camera's 30 FPS sampling rate limitations.
2. It injects known physical priors, forcing the neural network to only learn the residual interactions, drastically accelerating training convergence.
3. It provides a continuous gradient landscape, allowing an AI agent to iteratively optimize initial conditions (action vectors) via gradient ascent on an empirically derived reward

function.

The computational demands of integrating these continuous paths forward and backward across thousands of game events necessitate strict optimization of parallel workloads and algorithmic differentiation.

2. Dataset and Data Engineering Pipeline

The framework consumes raw event logs and high-frequency (30 FPS) spatial tracking streams of the puck and 10 skaters from the Stathletes Big Data Cup (2026). To prepare the data for the SciML pipeline, ZoneOpt.jl executes a robust preprocessing phase.

2.1 Synchronization and Trajectory Generation

Discrete play-by-play timestamps map directly to corresponding temporal indices in the Cartesian tracking space. The framework isolates 5-second continuous trajectory windows immediately following each zone entry to serve as training labels. To handle missing frames inherent to optical tracking, the framework applies a linear interpolation sequence, computing (1 - α) * left_value + α * right_value to bridge gaps.

To construct the full state matrix for the differential equation, velocities must be derived from the interpolated positional coordinates. The finite_difference function calculates continuous velocities (v_x, v_y, v_z) from the positional arrays (x, y, z) utilizing the time delta step dt. Time arrays are normalized relative to the sequence start, adjusting for dynamic frame rates by taking the median estimated FPS of the sequence.

2.2 Context Vectorization

A Neural ODE mapping real-world sports dynamics requires contextual awareness of external variables. The framework derives a static, 23-dimensional context vector C representing the spatial state of the ice at the exact moment of entry (t=0). This context vector is injected alongside the neural weights during the forward pass. The structure of the 23-dimensional vector is defined as follows:

- Indices 1 through 3 contain a one-hot representation of the categorical tactical strategy (Carried, Dumped, or Played).
- Indices 4 through 13 contain the puck-relative x and y coordinate configurations of the 5 attacking teammates.
- Indices 14 through 23 contain the puck-relative x and y coordinate configurations of the 5 defending opponents.

3. Universal Differential Equation (UDE) Formulation

Instead of relying on an opaque deep neural network attempting to guess sequential coordinates, the physics-informed state transitions follow a derivative modeled by a Universal Differential

Equation. The continuous state of the puck u(t) represents [x, y, z, v_x, v_y, v_z].

The governing equation is formulated as: $du/dt = f(u,p,t) + NN(u, C, \theta)$

3.1 Physics Priors

The function f(u,p,t) represents the hard-coded physics prior. We mathematically assert that the system experiences baseline kinematic drag and damping. Specifically, the prior enforces a drag coefficient of 0.12 against the x and y velocities, and a vertical damping factor of 0.30 coupled with a -0.05 gravity-like acceleration against the z velocity.

3.2 Neural Residual Estimation

The term $NN(u, C, \theta)$, parameterized by neural weights θ and context C, functions as the nonlinear estimator. It captures what basic equations miss (e.g., the puck bouncing off the boards, an opponent intercepting the trajectory, or unexpected friction variances). The neural architecture is built dynamically based on hyperparameters, defaulting to an input dimension of 30 (6 state variables + 23 context variables + 1 time variable), followed by multiple hidden layers utilizing tanh activations, and mapping to a 3-dimensional output corresponding to acceleration residuals. To ensure numerical stability within the solver, a normalization layer applies shifting and scaling based on dataset variance to the raw inputs before they enter the neural network.
This Universal Differential Equation is solved numerically via OrdinaryDiffEq.jl leveraging the Tsit5() (Tsitouras 5/4 Runge-Kutta) explicit solver, which provides an efficiency profile for non-stiff systems.

4. Adjoint Sensitivity Analysis and Algorithmic Differentiation

A key part of this project lies in how the neural network weights θ are optimized. Backpropagation through time (BPTT) for a standard discrete sequence scales memory linearly with the number of time steps. In a high-frequency ODE setting, tracking the tape across hundreds of intermediate solver steps is overwhelmingly inefficient.

To circumvent this, ZoneOpt.jl invokes the continuous Adjoint Method via the SciMLSensitivity.jl library. Through Pontryagin's Maximum Principle, an augmented state containing the adjoint vector is defined and solved backwards in time.

4.1 Implementation of Differentiation

The framework specifically utilizes InterpolatingAdjoint(autojacvec=ReverseDiffVJP(true)). This algorithm allows the solver to build a continuous interpolation spline of the forward pass, avoiding complete integration of the forward ODE during the backward pass. This creates a balance of compute and memory, yielding O(1) memory backpropagation. Reverse-mode automatic differentiation is then utilized locally to evaluate the Vector-Jacobian Products for the

Neural Network terms.

5. Parallelization and Computational Architecture

Training over spatial tracking data requires parallel computing considerations. ZoneOpt.jl optimizes the batching workload utilizing data parallelism principles covered in the class.

5.1 Data Parallelism and Batching

The loss evaluation requires solving the UDE across multiple discrete puck entry scenarios sequentially. The framework generates random batches of trajectories based on a configurable batch size, defaulting to 16 sequences per batch. By wrapping the ODE within an EnsembleProblem executing via EnsembleThreads() (or EnsembleDistributed), we farm the trajectory simulations mapping the batch loss across available Julia CPU threads. This aligns with multi-core processor usage, efficiently circumventing Julia's global boundaries where memory allocations stay isolated to specific thread heaps.

5.2 Objective Function and Optimization

The batch objective function evaluates the loss by comparing the interpolated sequence u(t) points to the empirical ground-truth tracking data. The total loss combines weighted positional and velocity errors. By default, the L2 position error carries a weight of 1.0, and the velocity error carries a weight of 0.25. The framework also supports a composite strategy quality loss penalty, scaled by automatically calculated class weights depending on successful shot distributions.

Optimization is driven by the Adam optimizer with an initial learning rate of 1e-3. The training loop incorporates a dynamic learning rate decay factor of 0.5 triggered after a set patience of epochs without validation improvement, alongside early stopping protocols to prevent overfitting.

6. Agentic Optimization Layer

With the SciML proxy environment established and trained, an AI tactical agent algorithm executes continuous policy evaluations. Given a fixed defensive context vector C, the agent tests multiple initial vectors (u0) to simulate strategies (Carried, Dumped, Played, Shot).

6.1 Tactical Reward Function

To evaluate the success of a simulated entry, the agent utilizes a custom continuous reward function spanning three spatial criteria:

1. **Zone Score:** Rewards pushing the puck deep into the offensive zone. It utilizes a logistic function scaling the x-coordinate, peaking

optimally around x ≈ 50 feet.

2. **Danger Score:** Rewards keeping the puck laterally close to the net. It applies a logistic curve penalizing distance from y ≈ 0, effectively prioritizing a highly dangerous central channel within ± 10 feet.
3. **Control Score:** Rewards moderate speeds to prevent the puck from exceeding player control thresholds, peaking at roughly 1 ft/s.

6.2 Performance Duality: ForwardDiff vs. Reverse-Mode

While reverse-mode AD (Zygote / ReverseDiff + Adjoint) is mathematically optimal computationally for training millions of Neural ODE parameters (θ), the agentic optimization layer presents a different derivative landscape. The AI system seeks to find the best immediate trajectory by solely optimizing the initial physical velocity of the puck (v_x,0, v_y,0) for a single scenario.

Because the input dimension of u0 is extraordinarily small (only 6 scalar values), reverse-mode AD presents unnecessary and heavily allocating overhead. The framework can then pivot to **Forward-Mode Differentiation (ForwardDiff.jl)** utilizing Dual Numbers. The model maps Jacobians pushing forward derivations of u0 simultaneously with primitive physics floats to discover the exact action vector that maximizes the empirical reward surface.

7. Results and Evaluation

Over the course of model training (recorded across 210+ epochs), multi-threading successfully condensed raw computing times spanning thousands of parameter evaluations.

- **Physics-Informed Accuracy:** The inclusion of kinematic priors resulted in the Neural ODE demonstrating exceptionally realistic energy dissipation. Unrealistic paths, such as pucks dramatically accelerating without corresponding player contextual momentum, were inherently reduced by the physics block.
- **Optimization Stability:** The training loss and trajectory RMSE decreased significantly without exploding gradients, a common catastrophic failure mode when raw discrete networks attempt to model continuous physical momentum.
- **Decision Validity:** Beyond pure coordinate RMSE, model utility was gauged using tactical validity. The evaluate_decision_validity function filters the test set for historical sequences that successfully resulted in a shot. Applying the trained environment to these historical successes, the simulated optimal path recommended by the AI correlated remarkably with the recorded real-world tactical decision. The recommendation output far superseded naive randomized baselines (~33%).

8. Limitations and Future Work

Several clear expansion pathways exist for ZoneOpt.jl:

1. **Local Minima inside Reward Surfaces:** The optimization of local velocity vectors currently faces highly non-convex topography driven by the rigid geometries of defending player coordinates. Incorporating simulated annealing or stochastic global optimization techniques before descending into local gradient ascent with ForwardDiff may yield stronger global minimum strategies.
2. **Defensive Velocity Derivatives:** The current static context state relies purely on the absolute positions at t=0. Providing the ODE access to defensive velocity approximations (the x' and y' derivatives of the players) as secondary context inputs will vastly strengthen the neural network's internal prediction of intercept trajectories.
3. **GPU Deployment:** The workflow primarily executes across standard CPU threads natively in Julia. Given the structure of the data and independent trajectories, integrating DiffEqGPU.jl in future iterations will map the EnsembleProblem batches directly onto CUDA acceleration hardware for hyper-scalar processing.

9. Conclusion

ZoneOpt.jl bridges the gap between discrete mechanical data-fitting and true continuous physics-engine simulation. By engineering a framework around a Universal Differential Equation optimized with Julia-based tools (OrdinaryDiffEq.jl, ReverseDiff.jl, SciMLSensitivity.jl, ForwardDiff.jl), the project successfully establishes a differential proxy environment capable of resolving the complexities of spatial sports analytics. Combining multi-threaded trajectory ensembles with adjoint-enhanced backward passes produces an AI system capable of rendering deterministic, optimized coaching decisions rooted directly in modeled physical reality.